Lists and Indexing

A value that holds other values

Gregory S. DeLozier, PhD

Breaking Changes

`length`, `slice`, `head`, and `tail` are keywords.

`[` and `]` are now reserved for list literals and indexing.

List Literals

```
empty = [];
mixed = [1, "two", true];
```

Elements do not need to share a type.

Reading by Index

```
numbers = [10, 20, 30];
print(numbers[1]);
```

Prints `20`. An index is a position, not the element.

Index Typing

```
if type(index) is bool or type(index) not in (int, float):
    raise TypeError("Index must be an integer")
if type(index) is float and not index.is_integer():
    raise TypeError("Index must be an integer")
```

`3.0` is valid. `3.5` and `true` are not.

Chained Indexing

```
matrix = [[1, 2], [3, 4], [5, 6]];
print(matrix[1][0]);
```

Prints 3. No special two-argument form.

Traversal with While and Length

```
values = [3, 1, 4, 1, 5, 9];
total = 0;
i = 0;
while (i < length(values)) {
    total = total + values[i];
    i = i + 1;
}
```

`total` ends at 23.

Slice, Head, and Tail

```
letters = [10, 20, 30, 40];
print(slice(letters, 1, 3));
print(head(letters));
print(tail(letters));
```

Prints `[20, 30]`, `10`, `[20, 30, 40]`.

Guarding Head and Tail

```
remaining = [3, 1, 2];
backwards = [];
while (length(remaining) > 0) {
    backwards = [head(remaining)] + backwards;
    remaining = tail(remaining);
}
```

`backwards` ends as `[2, 1, 3]`.

Parsing: List Literals and Postfix Chains

```
def parse_complex_suffixes(node, tokens):
    while tokens[0]["tag"] == "[":
        index, tokens = parse_expression(tokens[1:])
        tokens = require(tokens, "]", "Expected ']' after index")
        node = {"tag": "complex", "base": node, "index": index}
    return node, tokens
```

Evaluating an Index Read

```
if ast["tag"] == "complex":
    base, status = evaluate(ast["base"], environment)
    if status is not None:
        return base, status
    index, status = evaluate(ast["index"], environment)
    if status is not None:
        return index, status
    if type(base) not in (list, str):
        raise TypeError("Indexing requires a list or string")
    return base[require_index(index, base)], None
```

Indexed Assignment

```
assignment_statement ::= <identifier> { index_suffix } "=" expression
```

Same grammar helper as reading. The difference is only in what the evaluator does with the result.

Evaluating Indexed Assignment

```
elif target["tag"] == "complex":
    base, status = evaluate(target["base"], environment)
    if status is not None:
        return base, status
    index, status = evaluate(target["index"], environment)
    if status is not None:
        return index, status
    if type(base) is not list:
        raise TypeError("Indexed assignment requires a list")
    key = require_index(index, base)
    destination = base
```

The Evaluation Order

The target is resolved before the right-hand value, for every kind of target.

```
a = [1, 2, 3];
a[exit(6)] = exit(999);   # exit cannot actually nest like this;
                           # built directly as AST nodes to prove it
```

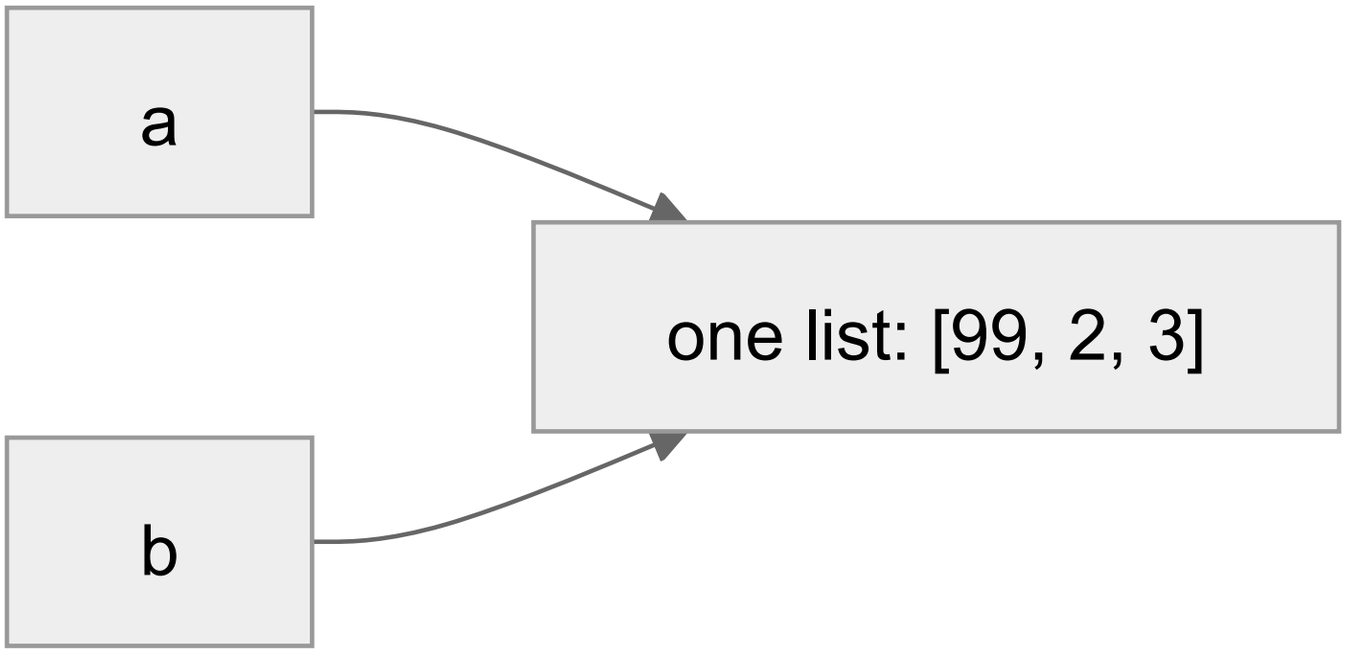
Returns `(6, "exit")`, not `(999, "exit")`.

Mutation Changes the Shared List

```
a = [1, 2, 3];
b = a;
b[0] = 99;
print(a);
print(b);
```

Both print `[99, 2, 3]`. Neither identifier was rebound.

Two Names, One List



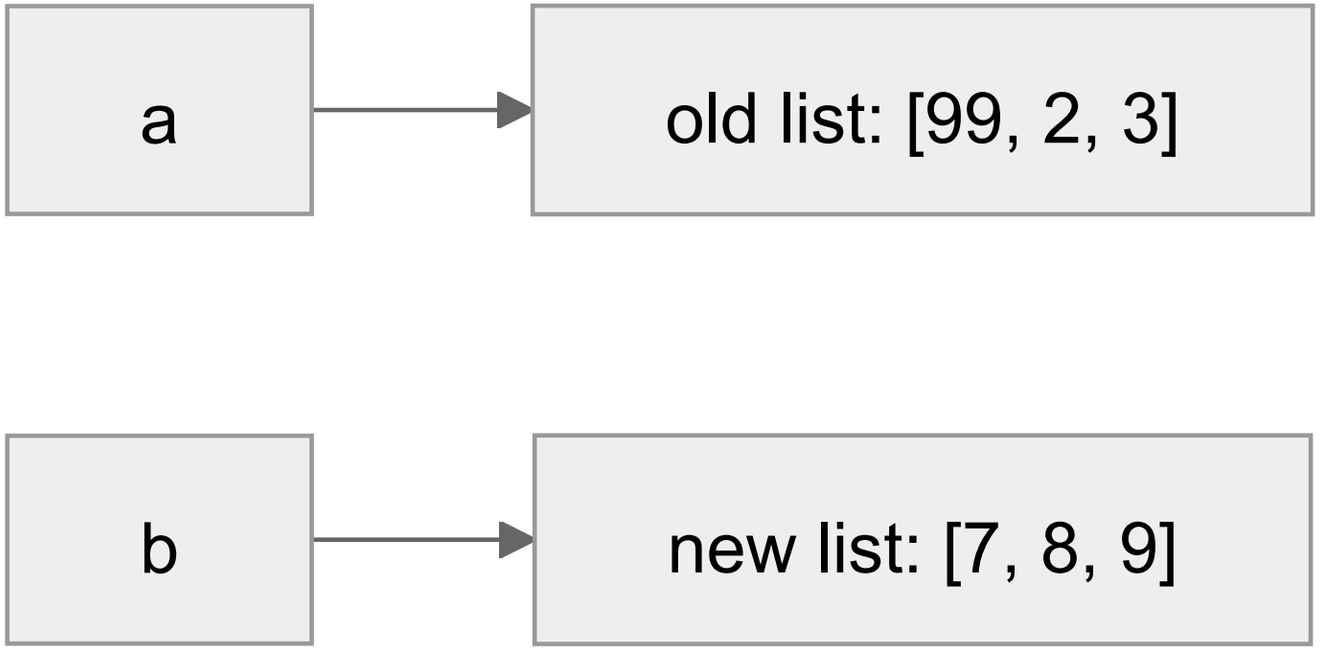
Changing the list is visible through both aliases.

Rebinding Changes One Identifier

```
// Continue from the shared-list example.
b = [7, 8, 9];
print(a);
print(b);
```

a prints `[99, 2, 3]`. b prints `[7, 8, 9]`.

Rebinding Redirects One Name



The old list stays unchanged. Only b's reference moved.

Mutation Reaches Outward

`x[10] = 4` reads `x` first, and reading has always searched outward through enclosing environments. Mutating what it finds needs no declaration.

Equality

```
print([1, 2] == [1, 2]);
print([true] == [1]);
```

Prints `true`, then `false`.

Concatenation

```
first = [1, 2];
second = [3, 4];
combined = first + second;
combined[0] = 99;
print(first);
```

Prints `[1, 2]`. `combined` is a new outer list.

Nested values remain shared, as with `slice` and `tail`.

Printing and Conversions

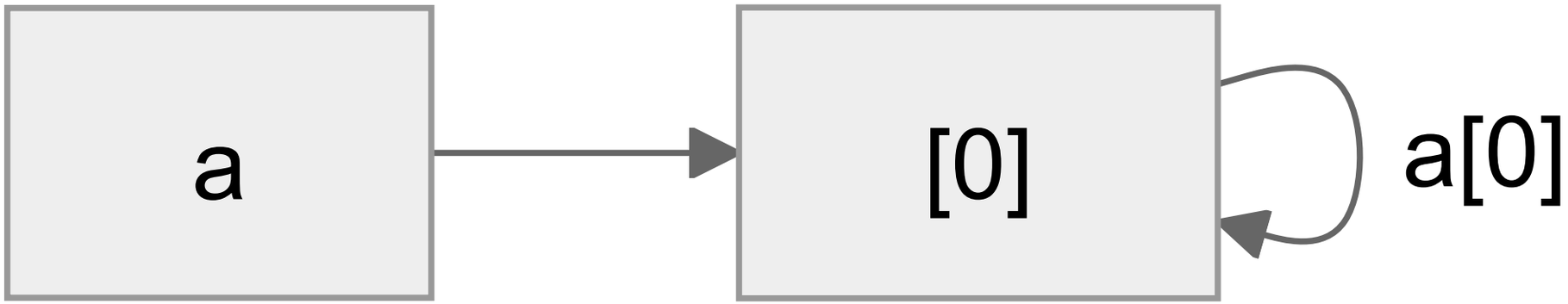
```
print([1, "hi", true, [2, 3]]);
```

Prints `[1, "hi", true, [2, 3]]`.

Self-Reference

```
a = [0];
a[0] = a;
```

`a[0]` is now the same list `a` names. Legal.



Self-Reference: Equality and Printing

Continuing from `a` on the previous slide:

```
print(a == a);
print(a);
```

Prints `true`, then `[[...]]`. No crash.

Testing the Contract

Empty and mixed-type literals; element evaluation order; nested literals and chained indexing.

Every rejected index case: fractional, boolean, string, negative, out of range.

Half-open slices, rejected bounds, head and tail on empty values, string length.

String indexing: reading a character, the same rejected indices, assignment into a string rejected.

Testing the Contract (continued)

Mutation through an alias versus rebinding leaving the alias untouched; structural equality including the nested true != 1 case.

Self-referential lists: direct and indirect cycles comparing equal to themselves and printing without crashing; two non-cyclic appearances of the same sublist still printing in full.

exit injected at every new position: a list element, an index base, an index, and an indexed assignment’s right-hand value.

Questions for Thought

Assignment shares a value. Duplicating a list requires a separate operation.

How can the scalar equality rule true != 1 affect the equality of two lists that look identical at a glance?

What state can change before an indexed assignment encounters exit, and what can only change after it?

head rejects an empty list; tail does not. What justifies treating the two differently?